CE152: Object-Oriented Programming

Unit 3

Dr. Sebastian Halder

CSEE

2026

Announcements

Marked exercise

- Exercise sheet is on Moodle in the **Lab Exercises (Week 16-18)** section
- Last chance to demonstrate is during the labs in **Week 18 (this week!)**
- Later demonstrations and demonstrations outside of the labs will not be possible
- If you are unable to demonstrate during this time contact the School Office and submit an **extenuating circumstances** claim
- If you do not demonstrate you will **receive a mark of zero**

Demonstration

- Read the instructions on the brief carefully
- **Practice beforehand**
- Open a project and a blank java file
- Notify GLA/supervisor you are ready
- You need to complete the task within 5 mins

The use of shortcuts

- The use of keyboard shortcuts is **allowed** for this exercise
- Clearly this excludes shortcuts such as *ctrl+v* to paste the solution from an external source

Academic support hour

- This week my support hours clash with a departmental meeting
- Please email me for a separate appointment

What are you supposed to learn (in this module)?

Python

```python
message = 'python is fun'
uppercase_message = message.upper()
print(uppercase_message)  # Output: PYTHON IS FUN
```

C Sharp

```
string str = "C# is fun";
string uppercaseStr = str.ToUpper();
Console.WriteLine(uppercaseStr);  // Output: C# IS FUN
```

Java

```java
String txt = "Java is best";
String upperCaseTxt = txt.toUpperCase();
System.out.println(upperCaseTxt);  // Output: JAVA IS BEST
```

Videos

Unit 3

- Please remember to watch the videos on **Encapsulation, Conditionals** and **Loops**
- You can find these videos on Moodle in **Unit 3: Encapsulation, Conditionals and Loops**
- The lecture that is taking place now is not identical to the videos

Unmarked Lab 2

Exercise 1B

Create a class called Exercise1B.

Add the following field:

- a String called s initialised with "University of Essex"

Add the following methods that return void and have no parameters:

- a method called `printLength()` that prints the length of `s`
- a method called `printFirstThree()` that prints the first three letters
- a method called `printFirstSpace()` that prints the position of the first space
- a method called `printUpperCase()` that prints `s` in upper case
- a method called `printWithoutSpaces()` that prints `s` without any spaces
- a method called `printNumberOfWords()` that prints the number of words in `s`

In this window paste your code for Exercise 1B without your main method.

Exercise 1B (1)

Create a class called Exercise1B.

Add the following field:

- a String called s initialised with "University of Essex"

```
public class Exercise1B {
  String s = "University of Essex";
}
```

Exercise 1B (2)

Add the following methods that return void and have no parameters:

- a method called `printLength()` that prints the length of `s`
- a method called `printFirstThree()` that prints the first three letters

```
public class Exercise1B {
  String s = "University of Essex";

  public void printLength() {
    System.out.println(s.length());
  }

  public void printFirstThree() {
      System.out.println(s.substring(0,3));
  }
}
```

Exercise 1B (3)

Add the following methods that return void and have no parameters:

- a method called `printFirstSpace()` that prints the position of the first space
- a method called `printUpperCase()` that prints `s` in upper case

```
public class Exercise1B {
  String s = "University of Essex";
  // previous methods removed to save space
  public void printFirstSpace() {
      System.out.println(s.indexOf(" "));
  }

  public void printUpperCase() {
      System.out.println(s.toUpperCase());
  }
}
```

Exercise 1B (4)

Add the following methods that return void and have no parameters:

- a method called `printWithoutSpaces()` that prints `s` without any spaces
- a method called `printNumberOfWords()` that prints the number of words in `s`

```
public class Exercise1B {
  String s = "University of Essex";
  // previous methods removed to save space
  public void printWithoutSpaces() {
      System.out.println(s.replaceAll(" ",""));
  }

  public void printNumberOfWords() {
      System.out.println(s.split(" ").length);
  }
}
```

Exercise 2A

- `Point2D` with `double` `x` and `y` and `distanceTo()` method
- `Triangle` extended from `Shape`, constructors for side lengths and `Point2D`, `area()` and `circumference()` methods

Exercise 2A: Point2D

```
public class Point2D {
    double x;
    double y;

    Point2D(double x, double y){
        this.x = x;
        this.y = y;
    }

    double distanceTo(Point2D other){
        return Math.sqrt(Math.pow(this.x-other.x,2)
          +Math.pow(this.y-other.y,2));
    }
}
```

Exercise 2A: Triangle Constructors

```java
public class Triangle extends Shape {
    double side1, side2, side3;

    Triangle(double side1, double side2, double side3){
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    Triangle(Point2D p1, Point2D p2, Point2D p3){
        this.side1 = p1.distanceTo(p2);
        this.side2 = p1.distanceTo(p3);
        this.side3 = p2.distanceTo(p3);
    }
}
```

Exercise 2A: Triangle Methods

```java
public class Triangle extends Shape {
    // constructors removed to save space
    @Override
    public double area(){
        double s = perimeter()/2;
        return Math.sqrt(s*(s-side1)*(s-side2)*(s-side3));
    }

    @Override
    public double perimeter(){
        return side1+side2+side3;
    }
}
```

Exercise 2B: Circle

```java
public class Circle extends Shape {
    double r;

    public Circle (double r){
        this.r = r;
    }

    public double area(){
        return Math.PI*Math.pow(r,2);
    }

    public double perimeter(){
        return 2*Math.PI*r;
    }
}
```

Quiz 2

Quiz summary

- Congratulations, more of you completed Quiz 2 than Quiz 1
- If you were not aware of the marked quizzes note there is one per week and completing it successfully will be worth 0.6% of your mark (6% in total if all 10 quizzes are successfully completed)

Question 9

- Create a non-public class called `Furniture`. This class should have a field of type `String` called `colour`.

```
class Furniture {
    String colour;
}
```

Question 9 (1)

- Create a non-public class called `Chair` that inherits from `Furniture`. This class should have a constructor with a parameter to initialize `colour`.

```
class Furniture {
    String colour;
}

class Chair extends Furniture {
    Chair(String colour) {
        this.colour = colour;
    }
}
```

Question 10

- Again create a class called `Furniture` with a field of type `String` called `colour`. This time add a constructor to `Furniture` that with a parameter of type `String` that you use to initialise `colour`.

```
class Furniture {
    String colour;

    Furniture (String colour){
        this.colour = colour;
    }
}
```

Question 10 (1)

- Again create a class called `Chair` that inherits from `Furniture` with a constructor that initialises `colour` via a parameter.

```
class Chair extends Furniture {
    Chair (String colour) {
        super(colour);
    }
}
```

Question 10 (1)

- The most common mistake was assigning the parameter in the constructor of chair directly to the field
- This leads to an error because the default constructor of Furniture was overridden (by adding a constructor with a parameter)

```
class Chair extends Furniture {
    // the most common mistake was this version:
    Chair (String colour) {
        this.colour = colour; // this is INCORRECT
    }
}
```

Thank you